Kurs: Linux + GitHub Actions, Teil 3 (v6)

Teil 3 baut auf Teil 1 und 2 auf. Du brauchst die Struktur deines eigenen Lern-Repositorys aus den vorherigen Teilen, insbesondere `daten/server.log` und `daten/zugriffe.log`. Für Teil 3 wird dasselbe Lern-Repository weiterverwendet; Git Flow ist dafür nicht nötig. Die Workflows verwenden `actions/checkout@v7`, `actions/upload-artifact@v7` und `actions/download-artifact@v8` auf `ubuntu-latest`. Am 06.11.2026 sind checkout v7.0.1, upload-artifact v7.0.2 und download-artifact v8.0.2 die neuesten veröffentlichten Patch-Versionen ihrer Hauptversionen.[1] Die Major-Tags folgen den Patch-Releases ihrer Hauptversion und pinnen keine einzelne Patch-Version. Alle drei Actions verwenden Node.js 24; selbst gehostete Runner benötigen dafür mindestens Version 2.327.1.[5] Es gibt **4 Lektionen** (9 bis 12) und ein Abschlussprojekt.

> Wie in Teil 2 gilt: Befehlsausgaben in den Erklärungen sind Beispiele. Prüfe sie gegen dein echtes Log im Actions-Tab.
>
> **Nummerierung:** In Teil 2 hieß der Auto-Prüfer `09-pruefer.yml`. Die neuen Dateien beginnen deshalb bei `10`. Die Lektionstitel zählen unabhängig davon (Lektion 9 bis 12).

Vorbereitung

Lege die gezeigten Workflow-Dateien unter ihren Pfaden auf dem Standardbranch deines Lern-Repositorys an und committe sie dort. So kannst du sie über **Actions** manuell starten.[11] Beim Bearbeiten im GitHub-Browser speichert **Commit changes** auf dem ausgewählten Branch; arbeitest du lokal, pushe deine Commits wie beschrieben. Lizenzhinweise in den Codeblöcken verweisen auf Dateien im Kurs-Repository.

[1] Offizielle Release-Seiten: https://github.com/actions/checkout/releases/tag/v7.0.1[2] , https://github.com/actions/upload-artifact/releases/tag/v7.0.2[3] und https://github.com/actions/download-artifact/releases/tag/v8.0.2[4] (neueste Patch-Versionen geprüft am 06.11.2026). Die Major-Tags `@v7` und `@v8` sind bewegliche Verweise auf die jeweils aktuelle Version ihres Hauptversionszweigs und pinnen keinen Patchstand.

[5] Die Metadaten der geprüften Releases nennen `node24`: https://github.com/actions/checkout/blob/v7.0.1/action.yml[6] , https://github.com/actions/upload-artifact/blob/v7.0.2/action.yml[7] und https://github.com/actions/download-artifact/blob/v8.0.2/action.yml[8] . Die checkout-Dokumentation nennt für Node.js-24-Actions mindestens Runner 2.327.1: https://github.com/actions/checkout/blob/v7/README.md[9] ; der Runner-Release bestätigt die Unterstützung von Node-24-Actions: https://github.com/actions/runner/releases/tag/v2.327.1[10] (geprüft am 06.11.2026).

[11] GitHub weist darauf hin, dass `workflow_dispatch` nur ausgelöst werden kann, wenn die Workflow-Datei auf dem Standardbranch liegt; der Lauf kann über den Actions-Tab gestartet und ein Branch ausgewählt werden: https://docs.github.com/en/actions/how-tos/manage-workflow-runs/manually-run-a-workflow[12] und https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows#workflow_dispatch[13] (geprüft am 06.11.2026).

Für Lektion 10 brauchst du außerdem einen Eintrag unter **Settings → Secrets and variables → Actions** (Beschreibung dort).[14][16]

Lektion 9: Matrix-Builds

Lernziel: Du erstellst mehrere sinnvolle Läufe aus einer Job-Definition, ergänzt mit `include` gezielt Werte und untersuchst, wie `fail-fast` mit Fehlern umgeht.

Action-Versionen: Am 06.11.2026 sind `checkout` v7.0.1, `upload-artifact` v7.0.2 und `download-artifact` v8.0.2 die verwendeten Patch-Versionen.[18] Die Major-Tags `@v7` und `@v8` folgen neuen Patch-Releases innerhalb ihrer Hauptversion und pinnen keine exakte Version. Alle drei Actions verwenden Node.js 24; selbst gehostete Runner benötigen dafür mindestens Version 2.327.1.[22] Der gehostete Runner `ubuntu-latest` wird von GitHub verwaltet.[28] Für produktive oder sicherheitskritische Workflows empfiehlt GitHub, Actions ohne Dependency-Lockfile auf eine geprüfte vollständige Commit-SHA zu pinnen und Aktualisierungen bewusst zu prüfen; ein Major-Tag ist einfacher zu warten, folgt aber einem beweglichen Verweis.[30]

Datei anlegen: `.github/workflows/10-matrix.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
```

[14] GitHub beschreibt das Anlegen von Repository-Secrets und deren Verwendung: https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets[15] (geprüft am 06.11.2026).

[16] GitHub dokumentiert Variablen auf Repository-, Organisations- und Umgebungsebene, die Sichtbarkeit von Konfigurationsvariablen sowie die `vars`-Syntax: https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-variables[17] (geprüft am 06.11.2026).

[18] Offizielle Release-Seiten: https://github.com/actions/checkout/releases/tag/v7.0.1[19] , https://github.com/actions/upload-artifact/releases/tag/v7.0.2[20] und https://github.com/actions/download-artifact/releases/tag/v8.0.2[21] (neueste Patch-Versionen geprüft am 06.11.2026). Die Major-Tags `@v7` und `@v8` sind bewegliche Verweise auf die jeweils aktuelle Version ihres Hauptversionszweigs und pinnen keinen Patchstand.

[22] Die Metadaten der geprüften Releases nennen `node24`: https://github.com/actions/checkout/blob/v7.0.1/action.yml[23] , https://github.com/actions/upload-artifact/blob/v7.0.2/action.yml[24] und https://github.com/actions/download-artifact/blob/v8.0.2/action.yml[25] . Die checkout-Dokumentation nennt für Node.js-24-Actions mindestens Runner 2.327.1: https://github.com/actions/checkout/blob/v7/README.md[26] ; der Runner-Release bestätigt die Unterstützung von Node-24-Actions: https://github.com/actions/runner/releases/tag/v2.327.1[27] (geprüft am 06.11.2026).

[28] GitHub beschreibt GitHub-gehostete Runner als von GitHub bereitgestellte virtuelle Maschinen und übernimmt deren Wartung und Aktualisierung: https://docs.github.com/en/actions/concepts/runners/github-hosted-runners[29] (geprüft am 06.11.2026).

[30] GitHub empfiehlt, Actions möglichst über Dependency-Lockfiles abzusichern; wenn das nicht möglich ist, wird das Pinning auf eine vollständige Commit-SHA als bewährte Maßnahme beschrieben. Tags können verschoben oder gelöscht werden: https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions#using-third-party-actions[31] (geprüft am 06.11.2026).

```
name: 10 - Matrix

on:
  workflow_dispatch:

jobs:
  suchen:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        suchbegriff: [INFO, WARNING, ERROR]
        datei: [daten/server.log]
        include:
          - suchbegriff: ERROR
            hinweis: "kritischer Fehler"
          - suchbegriff: produkte
            datei: daten/zugriffe.log
    steps:
      - uses: actions/checkout@v7

      - name: Einträge zählen
        env:
          SUCHBEGRIFF: ${{ matrix.suchbegriff }}
          DATEI: ${{ matrix.datei }}
          HINWEIS: ${{ matrix.hinweis }}
        run: |
          if [ ! -f "$DATEI" ]; then
            echo "Datei $DATEI fehlt" >&2
            exit 2
          fi
          if ANZAHL=$(grep -F -c -- "$SUCHBEGRIFF" "$DATEI"); then
            :
          else
            STATUS=$?
            if [ "$STATUS" -eq 1 ]; then
              ANZAHL=0
            else
```

```
    exit "$STATUS"
  fi
fi
echo "$SUCHBEGRIFF in $DATEI: $ANZAHL Einträge"
if [ -n "$HINWEIS" ]; then
  echo "$HINWEIS"
fi
```

Wie das funktioniert[32]

Konzept	Bedeutung
`strategy.matrix`	Erzeugt aus einer Job-Definition mehrere Jobs, einen pro Wert[34]
`matrix.suchbegriff`	Suchtext des jeweiligen Laufs; beim Zugriffslauf `produkte`
`matrix.datei`	Datei des jeweiligen Laufs
`include`	Ergänzt oder erweitert gezielt Matrix-Kombinationen[37]
`fail-fast: false`	Die übrigen Matrix-Läufe werden bei einem Fehler nicht wegen `fail-fast` abgebrochen[40]
Parallelität	GitHub führt Matrix-Jobs parallel aus, soweit Runner verfügbar sind[42]

[32] GitHub dokumentiert für `fail-fast`, dass bei einem fehlgeschlagenen Matrix-Job standardmäßig wartende und laufende Jobs abgebrochen werden; mit `false` unterbleibt dieses Abbruchverhalten: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategyfail-fast[33] (geprüft am 06.11.2026).

Im Actions-Tab erscheinen vier Jobs. Drei werten `server.log` nach INFO, WARNING und ERROR aus; der vierte zählt `produkte` in `zugriffe.log`. Die genaue Darstellung der Job-Namen kann sich in der Oberfläche ändern.

> Der Wert wird wieder über `env:` übergeben, nicht direkt in den Shell-Text eingesetzt. Das ist dieselbe Sicherheitsregel wie in Teil 1: `${{ ... }}` wird vor der Shell ersetzt, ein Wert mit Sonderzeichen könnte sonst den Befehl verändern. Bei festen Listenwerten wie hier wäre es ungefährlich, aber die Gewohnheit lohnt sich.[a]
>
> [a] GitHub empfiehlt für Inline-Skripte, nicht vertrauenswürdige Expressions zunächst in eine Umgebungsvariable zu übernehmen, statt sie unmittelbar in den Shell-Text einzusetzen: https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions#good-practices-for-mitigating-script-injection-attacks[44] (geprüft am 06.11.2026).

Die Shell-Behandlung unterscheidet drei Fälle: Eine fehlende Datei führt zu einem Fehler, kein Treffer ergibt den gültigen Zählerstand 0 und mindestens ein Treffer liefert die Trefferzahl. `grep -F` behandelt den Wert als wörtliche Zeichenfolge statt als regulären Ausdruck. Andere Suchfehler werden nicht verschluckt. `suchbegriff` beschreibt den gesuchten Text; auch ein `include`-Lauf darf daher einen anderen Suchbegriff wie `produkte` erhalten.[45]

[34] GitHub dokumentiert, dass Matrix-Konfigurationen alle Kombinationen der Dimensionen bilden und `include` Variablen zu kompatiblen Ursprungskombinationen hinzufügen oder, wenn keine Kombination ohne Überschreiben passt, neue Kombinationen ergänzt: https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs[35] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategymatrixinclude[36] (geprüft am 06.11.2026).

[37] GitHub dokumentiert, dass Matrix-Konfigurationen alle Kombinationen der Dimensionen bilden und `include` Variablen zu kompatiblen Ursprungskombinationen hinzufügen oder, wenn keine Kombination ohne Überschreiben passt, neue Kombinationen ergänzt: https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs[38] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategymatrixinclude[39] (geprüft am 06.11.2026).

[40] GitHub dokumentiert für `fail-fast`, dass bei einem fehlgeschlagenen Matrix-Job standardmäßig wartende und laufende Jobs abgebrochen werden; mit `false` unterbleibt dieses Abbruchverhalten: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategyfail-fast[41] (geprüft am 06.11.2026).

[42] GitHub beschreibt GitHub-gehostete Runner als von GitHub bereitgestellte virtuelle Maschinen und übernimmt deren Wartung und Aktualisierung: https://docs.github.com/en/actions/concepts/runners/github-hosted-runners[43] (geprüft am 06.11.2026).

[38] https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs

[39] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategymatrixinclude

[40] https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs

[41] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategymatrixinclude

[42] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategyfail-fast

[43] https://docs.github.com/en/actions/concepts/runners/github-hosted-runners

[45] GitHub dokumentiert, dass Matrix-Konfigurationen alle Kombinationen der Dimensionen bilden und `include` Variablen zu kompatiblen Ursprungskombinationen hinzufügen oder, wenn keine Kombination ohne

Mehrere Dimensionen

```
strategy:
  matrix:
    suchbegriff: [INFO, WARNING, ERROR]
    datei: [daten/server.log]
    include:
      - suchbegriff: produkte
        datei: daten/zugriffe.log
```

Der zugehörige Job-Code verwendet `grep` mit `matrix.suchbegriff` und `matrix.datei` wie im Beispiel der Datei `10-matrix.yml`.[48]

Das ergibt **4 sinnvolle Läufe**: drei Zählungen der Log-Stufen in `server.log` und eine Zählung von `produkte` in `zugriffe.log`. Bei Matrixdimensionen, die vollständig miteinander kombiniert werden, multiplizieren sich die Werte; `include` kann einzelne passende Kombinationen gezielt erweitern oder zusätzliche Läufe erzeugen.[51]

`include:` gezielt Kombinationen ergänzen

```
strategy:
  matrix:
    suchbegriff: [INFO, ERROR]
    include:
      - suchbegriff: ERROR
        hinweis: "kritischer Fehler"
```

Der Eintrag unter `include:` ergänzt die bestehende Kombination `suchbegriff: ERROR` um die zusätzliche Variable `hinweis` mit dem Wert „kritischer Fehler“. Sie steht als `matrix.hinweis` nur

Überschreiben passt, neue Kombinationen ergänzt: https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs[46] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategymatrixinclude[47] (geprüft am 06.11.2026).

[48] GitHub dokumentiert, dass Matrix-Konfigurationen alle Kombinationen der Dimensionen bilden und `include` Variablen zu kompatiblen Ursprungskombinationen hinzufügen oder, wenn keine Kombination ohne Überschreiben passt, neue Kombinationen ergänzt: https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs[49] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategymatrixinclude[50] (geprüft am 06.11.2026).

[51] GitHub dokumentiert, dass Matrix-Konfigurationen alle Kombinationen der Dimensionen bilden und `include` Variablen zu kompatiblen Ursprungskombinationen hinzufügen oder, wenn keine Kombination ohne Überschreiben passt, neue Kombinationen ergänzt: https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs[52] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategymatrixinclude[53] (geprüft am 06.11.2026).

für diese Kombination zur Verfügung. Es wird **kein zusätzlicher Lauf** erzeugt; die Anzahl bleibt bei zwei.[54] Ein `include`-Eintrag, der mit keiner ursprünglichen Kombination vereinbar ist, kann dagegen einen Lauf hinzufügen.[57]

Aufgaben 1. Starte den Workflow. Wie viele Jobs siehst du? Wie heißen sie? 2. Füge `FATAL` zur Liste `suchbegriff` hinzu. Läuft der Job grün durch? Warum? 3. Ändere den `include`-Eintrag mit `hinweis` vorübergehend von `suchbegriff: ERROR` zu `suchbegriff: WARNING`. Da WARNING bereits in der Ausgangsmatrix vorkommt, wird der bestehende WARNING-Lauf um den Hinweis erweitert; es bleibt bei vier Läufen. Starte den Workflow erneut und kontrolliere, in welchem Lauf der Hinweis ausgegeben wird. Stelle danach den ursprünglichen Wert wieder her. 4. Das Grundbeispiel enthält den `datei`-Eintrag und `include` bereits. Erkläre, warum diese vier Kombinationen fachlich sinnvoll sind, und nenne für jede Kombination Datei und Suchbegriff. 5. **Nur beobachten:** Lass den WARNING-Lauf bewusst fehlschlagen: Ergänze im `run:`-Block vor der Zählung `[ "$SUCHBEGRIFF" != "WARNING" ] || exit 1`. Starte mit `fail-fast: false` und kontrolliere, dass die anderen Matrix-Läufe weiterlaufen. Stelle danach `fail-fast: true` ein und beobachte, welche noch laufenden bzw. wartenden Läufe GitHub abbricht. Entferne die Fehlerzeile danach wieder. Das Timing ist nicht festgelegt und soll nicht bewertet werden.[60]

Lektion 10: Secrets und Variablen

Lernziel: Du unterscheidest nicht geheime Variablen von Secrets, verwendest beide sicher in einem Workflow und erklärst die Grenzen der Log-Maskierung.

Einrichten (einmalig):[62][64]

[54] GitHub dokumentiert, dass Matrix-Konfigurationen alle Kombinationen der Dimensionen bilden und `include` Variablen zu kompatiblen Ursprungskombinationen hinzufügen oder, wenn keine Kombination ohne Überschreiben passt, neue Kombinationen ergänzt: https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs[55] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategymatrixinclude[56] (geprüft am 06.11.2026).

[57] GitHub dokumentiert, dass Matrix-Konfigurationen alle Kombinationen der Dimensionen bilden und `include` Variablen zu kompatiblen Ursprungskombinationen hinzufügen oder, wenn keine Kombination ohne Überschreiben passt, neue Kombinationen ergänzt: https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs[58] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategymatrixinclude[59] (geprüft am 06.11.2026).

[60] GitHub dokumentiert für `fail-fast`, dass bei einem fehlgeschlagenen Matrix-Job standardmäßig wartende und laufende Jobs abgebrochen werden; mit `false` unterbleibt dieses Abbruchverhalten: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstrategyfail-fast[61] (geprüft am 06.11.2026).

[62] GitHub beschreibt das Anlegen von Repository-Secrets und deren Verwendung: https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets[63] (geprüft am 06.11.2026).

[64] GitHub dokumentiert Variablen auf Repository-, Organisations- und Umgebungsebene, die Sichtbarkeit von

1. Öffne im Repository **Settings → Secrets and variables → Actions**
2. Tab **Secrets** → **New repository secret**: Name `KURS_GEHEIMNIS`, Wert `Pinguin42`
3. Tab **Variables** → **New repository variable**: Name `KURS_THEMA`, Wert `Linux`

> **Nutze ausschließlich Testwerte.** `Pinguin42` ist ein reiner Demowert. Gib niemals echte Passwörter, Tokens oder Zugangsdaten ein, auch nicht vorübergehend. Das Beispiel prüft nur, ob das Secret gesetzt ist; es gibt weder den Wert noch seine Länge aus.

Datei anlegen: `.github/workflows/11-secrets.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 11 - Secrets und Variablen

on:
  workflow_dispatch:

jobs:
  zeigen:
    runs-on: ubuntu-latest
    steps:
      - name: Variable verwenden
        env:
          THEMA: ${{ vars.KURS_THEMA }}
        run: echo "Das Kursthema ist $THEMA"

      - name: Secret verwenden
        env:
          GEHEIMNIS: ${{ secrets.KURS_GEHEIMNIS }}
        run: |
          if [ -z "$GEHEIMNIS" ]; then
            echo "Secret fehlt oder ist leer"
            exit 1
          fi
          echo "Secret ist gesetzt"
```

Konfigurationsvariablen sowie die `vars`-Syntax: https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-variables[65] (geprüft am 06.11.2026).

Unterschied auf einen Blick

	Variable (`vars.`)[66]	Secret (`secrets.`)
Für	Nicht sensible Konfiguration	Passwörter, Tokens, Schlüssel
Sichtl in den Einst	Ja, Wert lesbar[68]	Nein, nach dem Speichern nicht mehr angezeigt[70]
Im Log	Wird normal angezeigt	GitHub versucht den Secret-Wert als *** zu maskieren; darauf nicht als Schutz verlassen[72]
Zugri	`${{ vars.NAME }}`	`${{ secrets.NAME }}`

Maskierung: eine Sicherung, kein verlässlicher Schutz

GitHub versucht, erkannte Secret-Werte im Log durch *** zu ersetzen. Das hilft gegen versehentliche Ausgabe, ist aber ausdrücklich keine Garantie.[74]

[66] GitHub dokumentiert Variablen auf Repository-, Organisations- und Umgebungsebene, die Sichtbarkeit von Konfigurationsvariablen sowie die `vars`-Syntax: https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-variables[67] (geprüft am 06.11.2026).

[68] GitHub dokumentiert Variablen auf Repository-, Organisations- und Umgebungsebene, die Sichtbarkeit von Konfigurationsvariablen sowie die `vars`-Syntax: https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-variables[69] (geprüft am 06.11.2026).

[70] GitHub dokumentiert, dass Secrets aus Fork-Pull-Requests (außer dem `GITHUB_TOKEN`) nicht an den Runner weitergegeben werden und gespeicherte Secret-Werte nach dem Speichern nicht angezeigt werden: https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets[71] (geprüft am 06.11.2026).

[72] GitHub weist darauf hin, dass Secret-Redaktion nicht garantiert ist, insbesondere bei transformierten Werten; die Dokumentation empfiehlt, veränderte sensible Werte nicht unmaskiert auszugeben: https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions#using-secrets[73] (geprüft am 06.11.2026).

[70] https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-variables

[71] https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-variables

[72] https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets

[73] https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions#using-secrets

[74] GitHub weist darauf hin, dass Secret-Redaktion nicht garantiert ist, insbesondere bei transformierten Werten; die Dokumentation empfiehlt, veränderte sensible Werte nicht unmaskiert auszugeben: https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions#using-secrets[75] (geprüft am 06.11.2026).

- Die Maskierung kann bei veränderten Werten (etwa kodiert, zerlegt oder umgeformt) versagen. Verlass dich nie darauf.[76]
- Secrets gehören nicht in Artefakte, Dateien oder Ausgaben, die andere sehen könnten.
- Gib Secrets nie mit `echo` aus. Auch bei einem Demowert üben wir hier nur die sichere Prüfung, ob das Secret gesetzt ist.
- Der Beispiel-Workflow gibt ausschließlich `Secret ist gesetzt` aus; er zeigt nicht den Wert und nicht dessen Länge.

Weitere Regeln zu Secrets - Bei Workflows, die durch einen `pull_request` aus einem **Fork** ausgelöst werden, stehen Secrets standardmäßig nicht zur Verfügung (Ausnahme: das `GITHUB_TOKEN`, dort mit eingeschränkten Rechten). Das schützt davor, dass fremder Code in einem Pull Request deine Secrets liest. Für andere Trigger und bestimmte Konfigurationen gelten abweichende Regeln.[78] - Der gespeicherte Wert wird in den Einstellungen nicht wieder angezeigt, du kannst ihn nur überschreiben. Berechtigte Workflows können ihn dennoch verwenden, sonst wären Secrets nutzlos.[80] - Secrets und Variablen gibt es auf Repository-, Organisations- und Umgebungsebene. Bei gleichem Namen gilt die spezifischere Ebene. Wie Umgebungen (`environments`) genau funktionieren, behandeln wir hier nicht.[82]

Aufgaben 1. Lege Secret und Variable an, starte den Workflow und vergleiche die Ausgaben. Was wird über das Kursthema und das Secret angezeigt? Bestätige, dass weder Secret-Wert noch Länge erscheinen. 2. Ändere die Variable `KURS_THEMA` in den Einstellungen und starte den Workflow erneut. Muss die Workflow-Datei geändert werden? 3. Erkläre ohne das Secret auszugeben, warum eine veränderte Darstellung (zum Beispiel kodiert oder umgedreht) möglicherweise nicht maskiert wird. Warum darfst du dich auf die Maskierung nicht verlassen? 4. Erkläre in zwei Sätzen: Warum sollte ein echtes Passwort nie als Variable gespeichert werden, auch wenn es technisch funktionieren würde?

---

[76] GitHub weist darauf hin, dass Secret-Redaktion nicht garantiert ist, insbesondere bei transformierten Werten; die Dokumentation empfiehlt, veränderte sensible Werte nicht unmaskiert auszugeben: https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions#using-secrets[77] (geprüft am 06.11.2026).

[78] GitHub dokumentiert, dass Secrets aus Fork-Pull-Requests (außer dem `GITHUB_TOKEN`) nicht an den Runner weitergegeben werden und gespeicherte Secret-Werte nach dem Speichern nicht angezeigt werden: https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets[79] (geprüft am 06.11.2026).

[80] GitHub dokumentiert, dass Secrets aus Fork-Pull-Requests (außer dem `GITHUB_TOKEN`) nicht an den Runner weitergegeben werden und gespeicherte Secret-Werte nach dem Speichern nicht angezeigt werden: https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets[81] (geprüft am 06.11.2026).

[82] GitHub dokumentiert Variablen auf Repository-, Organisations- und Umgebungsebene, die Sichtbarkeit von Konfigurationsvariablen sowie die `vars`-Syntax: https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-variables[83] (geprüft am 06.11.2026).

Lektion 11: Wiederverwendbare Workflows

Lernziel: Du rufst einen Workflow mit Eingaben auf und verfolgst, wie eine Ausgabe vom Schritt über Job und Workflow an den Aufrufer gelangt.

Idee: Statt dieselben Schritte in vielen Workflows zu kopieren, schreibst du sie einmal und rufst sie auf.

Datei anlegen 1: wiederverwendbarer Workflow `.github/workflows/12-zaehler.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 12 - Zähler (wiederverwendbar)

on:
  workflow_call:
    inputs:
      datei:
        description: "Pfad zur Datei"
        required: true
        type: string
      suchwort:
        description: "Gesuchtes Wort"
        required: true
        type: string
    outputs:
      anzahl:
        description: "Anzahl der Treffer"
        value: ${{ jobs.zaehlen.outputs.anzahl }}

jobs:
  zaehlen:
    runs-on: ubuntu-latest
    outputs:
      anzahl: ${{ steps.suche.outputs.anzahl }}
    steps:
      - uses: actions/checkout@v7

      - name: Zählen
```

```
        id: suche
        env:
          DATEI: ${{ inputs.datei }}
          SUCHWORT: ${{ inputs.suchwort }}
        run: |
          if [ ! -f "$DATEI" ]; then
            echo "Datei nicht gefunden: $DATEI" >&2
            exit 2
          fi
          if ANZAHL=$(grep -F -c -- "$SUCHWORT" "$DATEI"); then
            :
          else
            STATUS=$?
            if [ "$STATUS" -eq 1 ]; then
              ANZAHL=0
            else
              exit "$STATUS"
            fi
          fi
          echo "$SUCHWORT in $DATEI: $ANZAHL"
          echo "anzahl=$ANZAHL" >> "$GITHUB_OUTPUT"
```

Datei anlegen 2: aufrufender Workflow `.github/workflows/13-aufrufer.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 13 - Aufrufer

on:
  workflow_dispatch:

jobs:
  fehler:
    uses: ./.github/workflows/12-zaehler.yml
    with:
      datei: daten/server.log
      suchwort: ERROR
```

```
  zugriffe:
    uses: ./.github/workflows/12-zaehler.yml
    with:
      datei: daten/zugriffe.log
      suchwort: produkte

  ergebnis:
    runs-on: ubuntu-latest
    needs: [fehler, zugriffe]
    steps:
      - name: Ergebnisse ausgeben
        env:
          FEHLER: ${{ needs.fehler.outputs.anzahl }}
          ZUGRIFFE: ${{ needs.zugriffe.outputs.anzahl }}
        run: |
          echo "Fehler im Server-Log: $FEHLER"
          echo "Zugriffe auf /produkte: $ZUGRIFFE"
```

Wie das funktioniert

Element	Bedeutung
`on: workflow_call`	Macht den Workflow von anderen Workflows aufrufbar[84]
`inputs:`	Parameter, die der Aufrufer über `with:` übergibt[87]
`outputs:` (oben)	Werte, die der aufgerufene Workflow an den Aufrufer zurückgibt[90]
`uses: ./.github/workflows/...`	Ruft einen Workflow **im selben Repository** auf[93]
`with:`	Übergibt die Eingaben

Zwei Besonderheiten:[96]

- Ein Job, der einen Workflow aufruft, hat `uses:` **statt** `runs-on:` und `steps:`.
- Die Ausgabe wird in **mehreren Stufen** durchgereicht: Schritt → Job (`outputs:` am Job) → Workflow (`outputs:` unter `workflow_call`) → Aufrufer (`needs.<job>.outputs.<name>`).

> Das `--` in `grep -F -c -- "$SUCHWORT"` sagt `grep`, dass danach keine Optionen mehr kommen. So wird ein Suchwort, das mit `-` beginnt, nicht als Option gedeutet. `-F` sucht den Text wörtlich, nicht als regulären Ausdruck. Vorher prüft der Schritt, ob die Datei vorhanden ist; nur der Exit-Code 1 (kein Treffer) wird als Anzahl 0 behandelt.

> Der aufgerufene Workflow führt `actions/checkout` selbst aus. Das ist nötig, weil er in einem eigenen Job mit eigener Umgebung läuft.

Aufgaben 1. Starte `13-aufrufer.yml`. Wie erscheinen die aufgerufenen Jobs in der Graph-Ansicht? 2. Ergänze einen dritten Aufruf, der `WARNING` in `daten/server.log` zählt. Du änderst

[84] GitHub dokumentiert Eingaben, optionale Standardwerte, Outputs und lokale wiederverwendbare Workflows unter `workflow_call`: https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows[85] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call[86] (geprüft am 06.11.2026).

[87] GitHub dokumentiert Eingaben, optionale Standardwerte, Outputs und lokale wiederverwendbare Workflows unter `workflow_call`: https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows[88] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call[89] (geprüft am 06.11.2026).

[90] GitHub dokumentiert Eingaben, optionale Standardwerte, Outputs und lokale wiederverwendbare Workflows unter `workflow_call`: https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows[91] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call[92] (geprüft am 06.11.2026).

[93] GitHub dokumentiert Eingaben, optionale Standardwerte, Outputs und lokale wiederverwendbare Workflows unter `workflow_call`: https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows[94] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call[95] (geprüft am 06.11.2026).

[88] https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows

[89] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call

[90] https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows

[91] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call

[92] https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows

[93] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call

[94] https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows

[95] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call

[96] GitHub dokumentiert Eingaben, optionale Standardwerte, Outputs und lokale wiederverwendbare Workflows unter `workflow_call`: https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows[97] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call[98] (geprüft am 06.11.2026).

dafür nur den Aufrufer, nicht `12-zaehler.yml`. 3. Mache `suchwort` optional: Setze in `12-zaehler.yml` **beide** Angaben `required: false` und `default: "ERROR"`. Ohne den Default wäre `suchwort` beim Auslassen leer. Rufe den Workflow einmal ohne `suchwort` auf. Welches Suchwort wird verwendet?[99] 4. Benenne in `12-zaehler.yml` die Schritt-ID `suche` in `ermitteln` um, aber ändere zunächst nicht den Verweis `steps.suche.outputs.anzahl` in den Job-`outputs`. Prüfe den Workflow und die Ausgabe im aufrufenden Workflow. Passe anschließend den Verweis auf `steps.ermitteln.outputs.anzahl` an. Was war die Ursache des fehlenden Werts?

Lektion 12: Bedingungen, Zusammenfassungen und Fehlerbehandlung

Lernziel: Du steuerst Schritte mit Bedingungen und Statusfunktionen, behandelst erwartete Fehler und schreibst eine Lauf-Zusammenfassung.

Datei anlegen: `.github/workflows/14-bedingungen.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 14 - Bedingungen

on:
  workflow_dispatch:
    inputs:
      modus:
        description: "Modus"
        type: choice
        options:
          - normal
          - fehler
        default: normal

jobs:
  pruefen:
```

[99] GitHub dokumentiert Eingaben, optionale Standardwerte, Outputs und lokale wiederverwendbare Workflows unter `workflow_call`: https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows[100] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call[101] (geprüft am 06.11.2026).

```
runs-on: ubuntu-latest
timeout-minutes: 5
steps:
  - uses: actions/checkout@v7

  - name: Immer
    run: echo "Dieser Schritt läuft normal"

  - name: Absichtlicher Fehler
    if: ${{ inputs.modus == 'fehler' }}
    run: |
      echo "Jetzt schlägt es fehl"
      exit 1

  - name: Nur bei Erfolg
    if: ${{ success() }}
    run: echo "Alles bisher in Ordnung"

  - name: Nur bei Fehler
    if: ${{ failure() }}
    run: echo "Etwas ist schiefgegangen"

  - name: Aufräumschritt (läuft immer)
    if: ${{ always() }}
    run: echo "Dieser kurze Schritt läuft unabhängig vom Ergebnis"

  - name: Abbruch prüfen
    if: ${{ cancelled() }}
    run: echo "Abbruch erkannt (falls Schritt noch startet)"

  - name: Nach Erfolg oder Fehler
    if: ${{ !cancelled() }}
    run: echo "Der Lauf wurde nicht abgebrochen"

  - name: Zusammenfassung
    if: ${{ always() }}
    env:
      MODUS: ${{ inputs.modus }}
```

```
          STATUS: ${{ job.status }}
        run: |
          {
            echo "## Ergebnis"
            echo "- Modus: $MODUS"
            echo "- Status: $STATUS"
          } >> "$GITHUB_STEP_SUMMARY"

  optional:
    runs-on: ubuntu-latest
    steps:
      - name: Darf scheitern
        continue-on-error: true
        run: exit 1

      - name: Läuft trotzdem
        run: echo "Der Fehler davor wurde toleriert"
```

Wie das funktioniert

Element	Bedeutung
`if:`	Schritt oder Job läuft nur, wenn die Bedingung wahr ist[102]
`success()`	Wahr, wenn alle vorherigen Schritte erfolgreich waren (das ist die **Standardbedingung** ohne `if:`)[104]
`failure()`	Wahr, wenn ein vorheriger Schritt fehlgeschlagen ist[106]
`always()`	Wird unabhängig vom Ergebnis der vorherigen Schritte ausgeführt, auch nach einer Abbruchanforderung (der Abbruch kann den Schritt später trotzdem beenden)[108]
`cancelled()`	Wahr, wenn der Lauf abgebrochen wurde; der Schritt mit dieser Bedingung zeigt das Muster, wird nach Abbruch aber nicht garantiert ausgeführt[110]

Element	Bedeutung
`!cancelled()`	Läuft nach Erfolg und nach Fehler, aber nicht nach einem Abbruch[112]
`continue-on-error: true`	Ein Fehler in diesem Schritt lässt den Job nicht scheitern (wie im Job `optional`)[114]
`timeout-minutes`	Bricht den Job nach der angegebenen Zeit ab[116]
`$GITHUB_STEP_SUMMARY`	Datei, deren Markdown-Inhalt als Zusammenfassung im Lauf erscheint[118]
`job.status`	Aktueller Status des Jobs (`success`, `failure` oder `cancelled`)[120]

Warum „Nur bei Erfolg“ im Fehlermodus übersprungen wird: Der Schritt „Absichtlicher Fehler“ schlägt fehl. Danach wird jeder Schritt, dessen Bedingung ohne Statusfunktion auskommt, nur ausgeführt, wenn bisher alles erfolgreich war.[122] „Nur bei Erfolg“ nutzt `success()` ausdrücklich und wird deshalb übersprungen, während `failure()` und `always()` greifen.

`cancelled()` selbst beobachten: Der Schritt „Abbruch prüfen“ zeigt, wie `cancelled()`

[102] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[103] (geprüft am 06.11.2026).

[104] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[105] (geprüft am 06.11.2026).

[106] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[107] (geprüft am 06.11.2026).

[108] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[109] (geprüft am 06.11.2026).

[110] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[111] (geprüft am 06.11.2026).

[112] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[113] (geprüft am 06.11.2026).

[114] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[115] (geprüft am 06.11.2026).

[116] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[117] (geprüft am 06.11.2026).

[118] GitHub beschreibt, dass Inhalte der Datei `GITHUB_STEP_SUMMARY` als Zusammenfassung auf der Übersichtsseite eines Workflow-Laufs angezeigt werden: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-commands#adding-a-job-summary[119] (geprüft am 06.11.2026).

[120] GitHub beschreibt die möglichen Werte des `job`-Kontexts als `success`, `failure` oder `cancelled`: https://docs.github.com/en/actions/reference/workflows-and-actions/contexts#job-context[121] (geprüft am 06.11.2026).

[112] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

[113] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

[114] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

[115] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

[116] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

[117] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

[118] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

[119] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

[120] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-commands#adding-a-job-summary

[121] https://docs.github.com/en/actions/reference/workflows-and-actions/contexts#job-context

[122] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[123] (geprüft am 06.11.2026).

geschrieben wird. Sein Start nach einem manuellen Abbruch ist nicht garantiert; das zeigen auch sein Name und seine Ausgabe. Ein manueller Abbruch kann Schritte überspringen, bevor diese Bedingung noch sichtbar ausgewertet wird.[124] Betrachte deshalb den Abbruchversuch als Beobachtung, nicht als Test mit garantiertem Schritt-Ergebnis.

`always()` oder `!cancelled()`? Nach einer Abbruchanforderung kann ein Schritt mit `always()` noch starten. Der Abbruch kann ihn aber anschließend beenden, ein Weiterlaufen ist nicht garantiert. `always()` eignet sich für kurze, sichere Aufräumschritte. Für Schritte, die nach einem Fehler weiterlaufen sollen, bei einem Abbruch aber gar nicht erst starten sollen, ist `!cancelled()` meist die passendere Bedingung. Das gilt besonders für Schritte, die lange dauern oder hängen könnten.[126]

Die Zusammenfassung ansehen: Öffne den fertigen Lauf, die Zusammenfassung erscheint oben auf der Übersichtsseite des Laufs.[128]

> **Eingaben bei `workflow_dispatch`:** `type: choice` zeigt beim manuellen Start ein Auswahlfeld. Eingaben erreichst du über `inputs.<name>`.[a] Wie in Teil 1 gilt: Bei Freitext-Eingaben immer über `env:` in die Shell geben, nie direkt in `run:` einsetzen. Hier ist `modus` eine feste Auswahl, wird aber trotzdem über `env:` gelesen.
>
> [a] GitHub dokumentiert, dass `workflow_dispatch`-Eingaben im `inputs`-Kontext verfügbar sind und `type: choice` eine auswählbare Option als Zeichenfolge bereitstellt: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_dispatchinputs[130] (geprüft am 06.11.2026).

Aufgaben 1. Starte den Workflow mit Modus `normal`. Welche Schritte laufen, welche werden übersprungen? 2. Starte ihn mit Modus `fehler`. Welche Schritte laufen jetzt? Wo erscheint die Zusammenfassung? 3. Entferne beim Schritt `Aufräumschritt (läuft immer)` das `if: ${{ always() }}`. Wie ändert sich das Verhalten im Fehlerfall? Stelle die Zeile anschließend wieder her. 4. **Nur beobachten:** Setze `timeout-minutes: 1` und füge einen Schritt `sleep 90` ein. Was passiert ungefähr nach einer Minute? Entferne die Änderung danach wieder. 5. **Nur beobachten:** Füge einen Schritt `sleep 60` ein, starte den Workflow und brich den Lauf während des Wartens manuell ab. Prüfe, ob `cancelled()`- und `!cancelled()`-Schritte noch ausgeführt werden. Das Ergebnis ist timingabhängig und kein festes Soll-Ergebnis. Entferne den Testschritt anschließend.

[124] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[125] (geprüft am 06.11.2026).

[126] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[127] (geprüft am 06.11.2026).

[128] GitHub beschreibt, dass Inhalte der Datei `GITHUB_STEP_SUMMARY` als Zusammenfassung auf der Übersichtsseite eines Workflow-Laufs angezeigt werden: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-commands#adding-a-job-summary[129] (geprüft am 06.11.2026).

Abschlussprojekt: Auswertungs-Pipeline

Lernziel: Du sammelst Ergebnisse mehrerer Matrix-Läufe als Artefakte und fasst sie in einem eigenen Schritt zusammen.

Verbinde Matrix, Artefakte und Zusammenfassung. Für die Action-Versionen und den Versionsstand gelten die Angaben aus Lektion 9. Die offiziellen Readmes beschreiben `pattern` zum Filtern mehrerer Artefakte und `merge-multiple`, um sie in dasselbe Zielverzeichnis zu entpacken.[131]

Datei anlegen: `.github/workflows/15-pipeline.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 15 - Pipeline

on:
  workflow_dispatch:

jobs:
  zaehlen:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        stufe: [INFO, WARNING, ERROR]
    steps:
      - uses: actions/checkout@v7

      - name: Zählen und speichern
        env:
          STUFE: ${{ matrix.stufe }}
        run: |
```

[131] Die Readmes beschreiben `pattern` zum Filtern mehrerer Artefakte, `merge-multiple` zum Entpacken in dasselbe Zielverzeichnis und die erforderliche Eindeutigkeit von Upload-Namen: https://github.com/actions/upload-artifact/blob/v7/README.md[132] und https://github.com/actions/download-artifact/blob/v8/README.md[133] (geprüft am 06.11.2026).

```
        if [ ! -f daten/server.log ]; then
          echo "Datei daten/server.log fehlt" >&2
          exit 2
        fi
        mkdir -p ergebnis
        if ANZAHL=$(grep -F -c -- "$STUFE" daten/server.log); then
          :
        else
          STATUS=$?
          if [ "$STATUS" -eq 1 ]; then
            ANZAHL=0
          else
            exit "$STATUS"
          fi
        fi
        echo "$STUFE=$ANZAHL" > "ergebnis/$STUFE.txt"
        echo "$STUFE: $ANZAHL"

    - name: Teilergebnis hochladen
      uses: actions/upload-artifact@v7
      with:
        name: ergebnis-${{ matrix.stufe }}
        path: ergebnis/

zusammenfassen:
  runs-on: ubuntu-latest
  needs: zaehlen
  if: ${{ !cancelled() }}
  steps:
    - name: Teilergebnisse laden
      id: laden
      continue-on-error: true
      uses: actions/download-artifact@v8
      with:
        pattern: ergebnis-*
        merge-multiple: true
        path: sammlung/
```

```
- name: Zusammenfassung schreiben
  env:
    LADEN: ${{ steps.laden.outcome }}
  run: |
    if [ "$LADEN" != "success" ]; then
      echo "Download fehlgeschlagen (Status: $LADEN)," \
        "Details im Log des Schritts 'Teilergebnisse laden'." \
        | tee -a "$GITHUB_STEP_SUMMARY"
      exit 1
    fi
    if ! ls sammlung/*.txt > /dev/null 2>&1; then
      echo "Download erfolgreich, aber keine Teilergebnis-Dateien" \
        "gefunden." | tee -a "$GITHUB_STEP_SUMMARY"
      exit 1
    fi
    {
      echo "## Log-Auswertung"
      echo ""
      for datei in sammlung/*.txt; do
        echo "- $(cat "$datei")"
      done
    } >> "$GITHUB_STEP_SUMMARY"
    cat sammlung/*.txt
```

Das Prinzip - Die **Matrix** erzeugt je Stufe einen Lauf, jeder lädt sein Teilergebnis als eigenes Artefakt hoch (Artefaktnamen müssen innerhalb eines Laufs eindeutig sein, deshalb `ergebnis-${{ matrix.stufe }}`) - Der Job `zusammenfassen` lädt mit `pattern` und `merge-multiple` alle Teilergebnisse in einen Ordner - `if: ${{ !cancelled() }}` startet die Zusammenfassung nach Erfolg **und** nach einem fehlgeschlagenen Matrix-Lauf, aber nicht, wenn jemand den Lauf abbricht[134]

Für diese Pipeline bleiben die Werte bei den drei Stufen INFO, WARNING und ERROR; `FATAL` wird in Aufgabe 2 als zusätzliche Matrix-Variante ergänzt.

Der Kontrollfluss im Zusammenfassungsschritt

[134] GitHub beschreibt Statusfunktionen wie `success()`, `failure()`, `always()` und `cancelled()` sowie `continue-on-error` und Job-Zeitlimits in der Workflow-Syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[135] (geprüft am 06.11.2026).

Der Download hat `continue-on-error: true`, damit der Folgeschritt auch dann noch läuft, wenn der Download scheitert. Das offizielle Readme zu `download-artifact@v8` beschreibt den Download passender Artefakte, legt aber nicht fest, dass ein Muster ohne Treffer zwingend einen Fehlerstatus liefert.[136] Der Workflow behandelt daher beide möglichen Ergebnisse sicher: Er prüft erst `outcome` und kontrolliert anschließend, ob `.txt`-Dateien vorhanden sind. Bei null hochgeladenen Artefakten kann die Meldung je nach Action-Verhalten entweder „Download fehlgeschlagen“ oder „Download erfolgreich, aber keine Teilergebnis-Dateien gefunden“ lauten; beide Fälle beenden den Zusammenfassungsschritt mit Fehler. Ein leeres Ergebnis wird nie als Erfolg gemeldet.

`outcome` und `conclusion`: zwei Ergebnisse pro Schritt[138]

Für einen Schritt mit `continue-on-error: true` gibt es zwei Angaben, die auseinanderfallen können:

Angabe	Bedeutung	Bei fehlgeschlagenem Download mit `continue-on-error`
`steps.laden.outcome`	Das tatsächliche Ergebnis des Schritts, **bevor** `continue-on-error` angewendet wird	`failure`[140]

[136] Das Readme von `download-artifact@v8` dokumentiert das Filtern mit `pattern`, macht aber keine verbindliche Aussage zum Fehlerstatus bei einem Muster ohne Treffer: https://github.com/actions/download-artifact/blob/v8/README.md[137] (geprüft am 06.11.2026). Das konkrete Verhalten bei null Treffern bleibt daher unsicher und muss als Beobachtung behandelt werden.

[138] GitHub beschreibt `steps.<step_id>.outcome` als Ergebnis vor `continue-on-error` und `steps.<step_id>.conclusion` als Ergebnis nach Anwendung dieser Einstellung: https://docs.github.com/en/actions/reference/workflows-and-actions/contexts#steps-context[139] (geprüft am 06.11.2026).

Angabe	Bedeutung	Bei fehlgeschlagenem Download mit `continue-on-error`
`steps.lade`	Das Ergebnis **nach** Anwendung von `continue-on-error`, so wie GitHub es für den weiteren Ablauf wertet	`success`[142]

Deshalb prüft der Workflow `outcome`: Nur dort ist der echte Fehler noch sichtbar. Mit `conclusion` würde der Folgeschritt einen gescheiterten Download für erfolgreich halten.

> **Grenze dieser Lösung:** `continue-on-error` toleriert **jeden** Fehler des Download-Schritts, nicht nur "keine Artefakte vorhanden", sondern auch ungültige Einstellungen oder Berechtigungsprobleme. Der Folgeschritt kann diese Fälle nicht sicher unterscheiden, er erkennt nur, dass der Download nicht erfolgreich war. Die tatsächliche Ursache steht im Log des Download-Schritts. Für einen Kurs ist das eine vertretbare Vereinfachung. In eigenen Workflows solltest du genauer prüfen, welche Fehler du wirklich tolerieren willst.

> **Dateikollisionen bei `merge-multiple`:** Mit `merge-multiple: true` landen die Dateien aller Artefakte im selben Ordner.[a] Hier ist das unproblematisch, weil jeder Matrix-Lauf eine anders benannte Datei erzeugt (`INFO.txt`, `WARNING.txt`, `ERROR.txt`). Würden zwei Läufe eine gleichnamige Datei hochladen, könnte eine die andere überschreiben.

[a] Die Readmes beschreiben `pattern` zum Filtern mehrerer Artefakte, `merge-multiple` zum Entpacken in

[140] GitHub beschreibt `steps.<step_id>.outcome` als Ergebnis vor `continue-on-error` und `steps.<step_id>.conclusion` als Ergebnis nach Anwendung dieser Einstellung: https://docs.github.com/en/actions/reference/workflows-and-actions/contexts#steps-context[141] (geprüft am 06.11.2026).

[142] GitHub beschreibt `steps.<step_id>.outcome` als Ergebnis vor `continue-on-error` und `steps.<step_id>.conclusion` als Ergebnis nach Anwendung dieser Einstellung: https://docs.github.com/en/actions/reference/workflows-and-actions/contexts#steps-context[143] (geprüft am 06.11.2026).

[142] https://docs.github.com/en/actions/reference/workflows-and-actions/contexts#steps-context

[143] https://docs.github.com/en/actions/reference/workflows-and-actions/contexts#steps-context

> dasselbe Zielverzeichnis und die erforderliche Eindeutigkeit von Upload-Namen: https://github.com/actions/upload-artifact/blob/v7/README.md[144] und https://github.com/actions/download-artifact/blob/v8/README.md[145] (geprüft am 06.11.2026).

> **Zu `download-artifact@v8`:** `pattern` und `merge-multiple` werden von dieser Action unterstützt.[a] Prüfe die Optionen trotzdem vor der Veröffentlichung gegen die aktuelle Dokumentation, da sich Optionen und Hauptversionen ändern können.
>
> [a] Die Readmes beschreiben `pattern` zum Filtern mehrerer Artefakte, `merge-multiple` zum Entpacken in dasselbe Zielverzeichnis und die erforderliche Eindeutigkeit von Upload-Namen: https://github.com/actions/upload-artifact/blob/v7/README.md[146] und https://github.com/actions/download-artifact/blob/v8/README.md[147] (geprüft am 06.11.2026).

Aufgaben 1. Starte die Pipeline. Wo findest du die Zusammenfassung, und welche Zahlen zeigt sie? 2. Ergänze `FATAL` in der `stufe`-Liste des Matrixjobs. Wie sieht die Zusammenfassung aus? 3. Baue den Zählschritt so um, dass er den wiederverwendbaren Workflow aus Lektion 11 verwendet statt eigenem `grep`. Welche Teile der Pipeline müssen sich ändern, welche nicht? (Tipp: Ein Aufruf per `uses:` kann ebenfalls in einer Matrix stehen, aber der Artefakt-Upload müsste dann im wiederverwendbaren Workflow stattfinden.) 4. Teste zwei Fehlerfälle. Füge die angegebene Testzeile in den Schritt `Zählen und speichern` ein und entferne sie nach dem Test wieder: - **a) Ein Lauf scheitert vor dem Upload:** Füge vor der Zählung `[ "$STUFE" != "WARNING" ] || exit 1` ein. Der WARNING-Lauf lädt nichts hoch. Kontrolliere im Log, welche Dateien heruntergeladen und in die Zusammenfassung aufgenommen wurden. - **b) Alle Läufe scheitern vor dem Upload:** Füge `exit 1` direkt nach `mkdir -p ergebnis` ein. Es wird nichts hochgeladen. Ergänze im Schritt `Zusammenfassung schreiben` die Ausgabe von `conclusion` und gib beide Werte am Anfang des Schritts aus. Der Workflow setzt bisher nur `LADEN`:

```yaml
        CONCLUSION: ${{ steps.laden.conclusion }}
```

Ergänze diese Zeile unter `LADEN:` bei `env:` und füge am Anfang von `run:` Folgendes ein:

```bash
      echo "outcome=$LADEN, conclusion=$CONCLUSION"
```

Prüfe, ob die Action einen Fehler meldet oder der Download-Schritt erfolgreich, aber ohne Dateien en
Ausgabe.

5. Breche einen laufenden Lauf von Hand ab. Beobachte, ob `zusammenfassen` startet oder weiterläuft. Ändere die Bedingung auf `always()` und wiederhole den Versuch. Die Ergebnisse hängen vom Abbruchzeitpunkt ab; bewerte nicht, ob der Job noch gestartet oder beendet wurde. Stelle die ursprüngliche Bedingung anschließend wieder her.

Häufige Fehler in Teil 3

Fehler	Ursache
Matrix-Wert ist leer	Falsche Schreibweise, es heißt `matrix.suchbegriff` und nicht `matrix.Suchbegriff` oder `${{ suchbegriff }}`; für die Datei nutze `matrix.datei`
Zu viele Läufe	Zwei Dimensionen multiplizieren sich
Secret fehlt oder ist leer	Name falsch geschrieben, nicht angelegt oder im Kontext nicht verfügbar
Secret erscheint lesbar im Log	Der Wert wurde ausgegeben oder verändert; Maskierung ist keine Garantie
`workflow_call` startet nicht	Pfad bei `uses:` falsch, Datei nicht auf dem Branch, `on: workflow_call` fehlt[148]
Output des Aufrufers leer	Ausgabe nicht in allen Stufen durchgereicht (Schritt, Job, Workflow)[151]
`uses:` zusammen mit `steps:`	Aufrufende Jobs haben `uses:` **statt** `runs-on:` und `steps:`[154]
`suchwort` ist leer trotz `required: false`	`default:` fehlt; `suchwort` ist eine Eingabe des wiederverwendbaren Workflows, `suchbegriff` dagegen ein Matrix-Wert[157]
Schritt nach Fehler übersprungen	Standardmäßig gilt `success()`, `always()`, `!cancelled()` oder `failure()` nutzen
Schritt nach Abbruch wird nicht ausgeführt	Auch mit `cancelled()` oder `always()` ist ein Start nach manueller Abbruchanforderung nicht garantiert

Fehler	Ursache
Artefakt-Download findet nichts	Muster passt nicht zu den Artefaktnamen, alle Uploads sind gescheitert, oder die Artefakte kommen aus einem anderen Lauf[160]
Download-Fehler wird nicht erkannt	`conclusion` statt `outcome` geprüft, oder `continue-on-error` verbirgt die Ursache (im Log des Download-Schritts nachsehen)[162]
Zwei Artefakte mit gleichem Namen	Artefaktnamen müssen innerhalb eines Laufs eindeutig sein[164]
`grep`-Zähler ist leer oder unerwartet	Datei prüfen; nur Status 1 bedeutet „kein Treffer“, andere Fehler weitergeben
Dateien überschreiben sich nach dem Download	`merge-multiple` legt alle Dateien in einen Ordner, gleichnamige Dateien kollidieren[167]

[148] GitHub dokumentiert Eingaben, optionale Standardwerte, Outputs und lokale wiederverwendbare Workflows unter `workflow_call`: https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows[149] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call[150] (geprüft am 06.11.2026).

[151] GitHub dokumentiert Eingaben, optionale Standardwerte, Outputs und lokale wiederverwendbare Workflows unter `workflow_call`: https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows[152] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call[153] (geprüft am 06.11.2026).

[154] GitHub dokumentiert Eingaben, optionale Standardwerte, Outputs und lokale wiederverwendbare Workflows unter `workflow_call`: https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows[155] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call[156] (geprüft am 06.11.2026).

[157] GitHub dokumentiert Eingaben, optionale Standardwerte, Outputs und lokale wiederverwendbare Workflows unter `workflow_call`: https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows[158] und https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call[159] (geprüft am 06.11.2026).

[160] Das Readme von `download-artifact@v8` dokumentiert das Filtern mit `pattern`, macht aber keine verbindliche Aussage zum Fehlerstatus bei einem Muster ohne Treffer: https://github.com/actions/download-artifact/blob/v8/README.md[161] (geprüft am 06.11.2026). Das konkrete Verhalten bei null Treffern bleibt daher unsicher und muss als Beobachtung behandelt werden.

[162] GitHub beschreibt `steps.<step_id>.outcome` als Ergebnis vor `continue-on-error` und `steps.<step_id>.conclusion` als Ergebnis nach Anwendung dieser Einstellung: https://docs.github.com/en/actions/reference/workflows-and-actions/contexts#steps-context[163] (geprüft am 06.11.2026).

[164] Die Readmes beschreiben `pattern` zum Filtern mehrerer Artefakte, `merge-multiple` zum Entpacken in dasselbe Zielverzeichnis und die erforderliche Eindeutigkeit von Upload-Namen: https://github.com/actions/upload-artifact/blob/v7/README.md[165] und https://github.com/actions/download-artifact/blob/v8/README.md[166] (geprüft am 06.11.2026).

[167] Die Readmes beschreiben `pattern` zum Filtern mehrerer Artefakte, `merge-multiple` zum Entpacken in dasselbe Zielverzeichnis und die erforderliche Eindeutigkeit von Upload-Namen: https://github.com/actions/upload-artifact/blob/v7/README.md[168] und https://github.com/actions/download-artifact/blob/v8/README.md[169] (geprüft am 06.11.2026).

[156] https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows
[157] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call
[158] https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows
[159] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call
[160] https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows
[161] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call
[162] https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows
[163] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onworkflow_call
[164] https://github.com/actions/download-artifact/blob/v8/README.md
[165] https://docs.github.com/en/actions/reference/workflows-and-actions/contexts#steps-context
[166] https://github.com/actions/upload-artifact/blob/v7/README.md
[167] https://github.com/actions/download-artifact/blob/v8/README.md
[168] https://github.com/actions/upload-artifact/blob/v7/README.md
[169] https://github.com/actions/download-artifact/blob/v8/README.md

Spickzettel-Ergänzung: Eintrag in deine `CHEATSHEET.md`

Lege in deinem eigenen Lern-Repository eine Datei `CHEATSHEET.md` an, falls du sie noch nicht hast. Ergänze dort zum Beispiel diese Übersicht:

```
strategy.matrix                 Ein Job, mehrere Läufe
matrix.<name>                   Wert im jeweiligen Lauf, zum Beispiel `matrix.suchbegriff`
matrix.include                  Kombination ergänzen oder erweitern
fail-fast: false                Andere Läufe nicht wegen eines Fehlers abbrechen
vars.NAME                       Variable (nicht geheim)
secrets.NAME                    Secret (nicht ausgeben)
on: workflow_call               Workflow aufrufbar machen
uses: ./.github/workflows/x.yml Lokalen Workflow aufrufen
with:                           Eingaben übergeben
if: ${{ !cancelled() }}         Läuft nach Erfolg und Fehler, nicht nach Abbruch
if: ${{ always() }}             Kann auch nach Abbruchanforderung starten
if: ${{ failure() }}            Läuft nur nach Fehler
if: ${{ cancelled() }}          Bedingung für abgebrochene Läufe
continue-on-error: true         Schrittfehler tolerieren, outcome bleibt sichtbar
steps.<id>.outcome              Echtes Ergebnis eines Schritts (vor continue-on-error)
steps.<id>.conclusion           Ergebnis nach continue-on-error (Fehler wird zu success)
timeout-minutes: N              Zeitlimit für den Job
>> "$GITHUB_STEP_SUMMARY"       Zusammenfassung im Lauf anzeigen
```
